

学校代码：10022

本科毕业论文(设计)

面向工具调用智能体的安全防火墙 Web 应用系统设计与实现

**Design and Implementation of a Web-based Agent Firewall for Tool-
Calling AI Systems**

霍玮放

学 院	工学院
专 业	电气工程及其自动化（辅修）
班 级	电气 24-3
学 号	230206108
指导教师	杨波 教授

2026 年 5 月

独创性声明

本人声明所呈交的论文（设计）是本人在导师指导下独立进行的设计、研究工作及取得的设计、研究成果。尽我所知，除了论文（设计）中特别加以标注和致谢的地方外，论文（设计）中不包含其他人已经发表或撰写过的研究成果，本论文（设计）中没有抄袭他人研究成果和伪造数据等行为。与我共同工作的人员对本研究所做的任何贡献均已在论文（设计）中作了明确的说明并表示了谢意。

作者签名： 日期： 2026 年 月 日

关于毕业论文（设计）使用授权的说明

本人完全了解北京林业大学有关保留、使用毕业论文（设计）的规定，即：本科生在校期间毕业论文（设计）工作的知识产权单位属北京林业大学；学校有权保留并向国家有关部门或机构送交论文（设计）的纸质版和电子版，允许毕业论文（设计）被查阅、借阅和复印；学校可以将毕业论文（设计）的全部或部分内容公开或编入有关数据库进行检索，可以允许采用影印、缩印或其它复制手段保存、汇编毕业论文（设计）。

（保密的论文在解密后应适用本授权书）

作者签名： 指导老师签名：

日 期： 2026 年 月 日

北京林业大学本科毕业论文(设计)AI 工具规范使用诚信承诺书

一、AI 工具使用情况说明

本人郑重声明，在毕业论文(设计)写作过程中使用 AI 工具的情况如下：

☐未使用任何 AI 工具(勾选此项者无需填写后续内容)

☒使用 AI 工具(勾选此项者需如实填写以下内容)

1.使用工具名称：ChatGPT、DeepSeek、论文写作规范辅助、文献数据库检索工具、LaTeX 排版与图表辅助工具。

2.具体用途及生成内容：用于文献检索关键词整理、论文结构草拟、学术语言润色建议、GB/T 7714-2015 参考文献格式检查、LaTeX 格式调整、图表生成脚本辅助编写。系统设计、实验方案、核心代码理解、数据复核、结论推导由作者结合系统实现独立完成。

3.生成内容占全文比例：25%。

二、诚信承诺

本人承诺：毕业论文(设计)核心研究内容(包括实验设计、数据分析、结论推导等)均独立完成，未使用 AI 工具直接生成或替代。上述 AI 工具使用情况描述真实、准确、完整，无隐瞒或虚假陈述。

学生签名： 日期： 2026..

摘要

大语言模型驱动的智能体正在从文本问答系统演进为能够接收外部消息、规划工具调用、访问本地技能并长期运行的自动化执行环境。Model Context Protocol (MCP) 和 OpenClaw 等工具生态降低了模型连接外部能力的成本，也使提示词注入、工具滥用、敏感凭据泄露、混淆代理和间接工具结果注入等风险进入真实运行链路。对个人或小团队部署的 OpenClaw 运行时而言，风险并不因为系统规模较小而降低：本机 skill 和 MCP provider 可能触达文件、账号凭据或第三方服务，工具返回内容又会重新进入模型上下文。仅依赖系统提示词约束模型行为，难以形成可证明的安全边界。

本文围绕 Agent-Firewall 系统，设计并实现了一个面向 OpenClaw 工具调用智能体的中间安全层 Web 应用系统。系统把 Agent-Firewall 定位为 OpenClaw 外侧的安全壳，而不是某个聊天入口本身：消息入口适配器（已实现 Telegram Bridge）把请求送入受保护运行时；Proxy Service 的/v1/scan 执行确定性输入扫描；Agent Runtime 顺序运行器组织模型响应、工具计划和审批状态；pre-tool gate 完成 RBAC、Schema、注入模式、预算和确认检查；工具执行统一分发到 Internal、OpenClaw 或 MCP provider；post-tool gate 对工具结果进行 PII、密钥和间接注入清洗；Trace/Audit 记录完整证据链并通过原入口返回结果。系统默认使用 uv、SQLite 和 memory cache，本地路径不依赖 Docker、PostgreSQL、Redis 或 Langfuse。

实验采用“离线确定性复核+本机联调”的设计：离线复核验证 Agent 层前置门控、后置清洗、运行图、Control Plane runtime spec 和 11 类 Agent 工具链回放案例；本机联调于 2026 年 5 月 8 日通过 allowlisted Telegram bot、OpenClaw provider 和 Agent-Firewall 本机服务完成脱敏验证，覆盖消息入口、Proxy 扫描、工具门控、人工审批、post-tool redaction 和 Trace 之间的闭环。实验整理得到红队场景资产共 358 个场景，其中 Playground 216 个、Agent 142 个；对应 pytest 参数化复测得到 470 个通过、125

个 xfail 和 78 个失败断言，用于界定轻量规则与扫描器组合尚未覆盖的语义伪装、凭据格式、多语言和多轮诱导边界。本机联调进一步证明，普通工具调用可被放行，高敏工具会进入 tool_confirmation 审批并在批准后携带 approved_intervention_id 重放，可疑输入会生成 input_block 且拒绝后不执行工具。联调结果同时表明，post-tool gate 能够在工具结果进入模型上下文前清洗 PII，但用户原始输入中已经出现的 PII 仍需要输出侧策略继续补强。

研究表明，将消息入口、Proxy 扫描、Agent 工具门控、人工审批和 Trace 审计组合为连续边界，可以在不禁止 OpenClaw/MCP 工具能力的前提下，降低个人智能体运行时的外部消息和工具调用风险。系统的局限在于本次轻量检测对语义伪装、低慢多轮攻击和工具描述供应链污染覆盖不足，后续可在工具描述签名、多轮风险累计、子 Agent 委派策略验证和生产级认证隔离方面继续完善。

关键词：智能体安全，工具调用，提示词注入，RBAC，OpenClaw

Abstract

LLM-based agents are evolving from text-only assistants into long-running execution environments that receive external messages, plan tool calls, and invoke local skills. Protocols and runtimes such as the Model Context Protocol (MCP) and OpenClaw make tool integration easier, but they also expose prompt injection, tool abuse, secret leakage, confused-deputy behavior, and indirect tool-result injection to operational systems. For a personal OpenClaw runtime, the risk remains concrete: skills and MCP providers may access local files or third-party services, and tool outputs can re-enter the model context. System prompts alone cannot be treated as a reliable security boundary.

This thesis designs and implements Agent-Firewall as a web-based safety shell around OpenClaw-enabled tool-calling agents. Message ingress adapters, including the currently implemented Telegram Bridge, send requests into the protected runtime. The proxy `/v1/scan` endpoint performs deterministic scan-only enforcement. The agent runner coordinates model responses, tool plans, and approval state. The pre-tool gate enforces RBAC, argument schemas, injection checks, budget limits, and confirmation requirements. Internal, OpenClaw, and MCP providers share the same execution hub. The post-tool gate redacts PII and secrets and blocks unsafe tool outputs before they can reach the LLM context. The default local deployment uses uv, SQLite, and an in-memory cache; Docker, PostgreSQL, Redis, and Langfuse are not required on the current local path.

The evaluation combines deterministic offline validation with a local Telegram-to-OpenClaw integration run on May 8, 2026. The offline validation covers pre-tool enforcement, post-tool sanitization, chat endpoint behavior, tool-role configuration, runtime-spec compatibility, and 11 agent-chain replay cases. The local validation uses an allowlisted Telegram bot, OpenClaw provider execution, high-sensitivity tool_confirmation, approved replay with approved_intervention_id, post-tool redaction, trace persistence, and rejected input-block handling. The current scenario inventory contains 358 red-team cases, including 216 Playground cases and 142 Agent cases. The pytest parameterized replay produced 470 passes, 125 xfail cases, and 78 failed assertions, making the remaining gaps in semantic disguise, credential formats, multilingual attacks, and multi-turn inducement explicit. The validation also shows that post-tool redaction protects tool outputs before they re-enter the model context, while PII already present in the user input still requires stronger output-side controls.

The results show that combining ingress control, proxy scanning, agent tool gates, human intervention, and structured tracing can reduce the blast radius of OpenClaw/MCP-enabled personal agents without disabling tool use entirely. Remaining limitations include semantic obfuscation, slow multi-turn attacks, tool-description supply-chain risk, and

production-grade isolation. Future work should strengthen signed tool descriptors, multi-turn risk accumulation, delegated-agent policy verification, authentication, and long-term trace governance.

Keywords: AI agent security, tool calling, prompt injection, RBAC, OpenClaw

目 录

摘要	I
Abstract	III
主要符号与缩略词对照表	VII
1 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	1
1.3 研究内容与论文结构	2
2 需求分析与威胁模型	4
2.1 系统目标与设计约束	4
2.2 资产与信任边界	4
2.3 攻击者模型	6
2.4 防护需求	6
3 系统总体设计	8
3.1 总体架构	8
3.2 Proxy Firewall scan-only 流水线	8
3.3 Agent Runtime 运行图	9
3.4 前端可视化与系统交互	10
4 核心模块实现	12
4.1 风险评分与决策聚合	12
4.2 RBAC 与角色继承	13
4.3 前置工具门控	13
4.4 后置工具门控	15
4.5 工具提供者与 OpenClaw/MCP 桥接	16
4.6 子智能体委派链追踪	16
4.7 Trace 审计与可解释性	17
5 实验设计与结果分析	19
5.1 实验环境与复现口径	19
5.2 Agent 层测试与本机联调	20
5.3 红队场景检测结果	21
5.4 轻量规则与完整扫描器能力对照	23
5.5 延迟开销分析	24
5.6 Agent 工具链持续调用与拦截案例	25
5.7 误报与漏报分析	26
6 总结与展望	27
6.1 主要工作总结	27
6.2 创新点	27
6.3 局限性	28
6.4 未来工作	28

参考文献..... 29

致谢 31

附录 A 复现实验命令 32

 A.1 Agent 层纯内存测试..... 32

 A.2 Agent 工具链离线回放..... 32

 A.3 红队场景离线复测..... 33

 A.4 执行耗时复核..... 33

 A.5 真机联调脱敏复核..... 33

附录 B 图表生成 35

附录 C 关键代码文件索引 37

主要符号与缩略词对照表

符号或缩略词	含义
Agent	具备自主规划、上下文记忆和工具调用能力的智能体运行单元。
LLM	Large Language Model，大语言模型。
Message ingress adapter	将外部消息送入受保护运行时的适配层；系统已实现 Telegram Bridge。
Telegram Bridge	可选消息入口适配器，负责 Telegram 轮询、白名单检查和审批后续执行。
Proxy Service	运行在 8000 端口的 FastAPI 服务，负责/v1/scan、审计日志、OpenClaw 发现、runtime spec 和审批队列。
Agent Runtime	运行在 8002 端口的 FastAPI 服务，负责受保护运行图、工具门控、OpenClaw/MCP 调用和 Trace 转发。
Control Plane	控制平面，负责生成运行时工具、角色、provider 引用和门控开关等配置。
Runtime spec	Control Plane 下发给 Agent Runtime 的运行时规格，包含允许工具、敏感度、provider 引用和门控配置。
Provider	工具能力提供者，本文包括 Internal、OpenClaw 和 MCP 三类来源。
MCP	Model Context Protocol，用于连接模型、上下文和工具的协议。
OpenClaw	具备技能、工具、文件系统和外部服务访问能力的智能体运行框架。
RBAC	Role-Based Access Control，基于角色的访问控制。
PII	Personally Identifiable Information，个人身份信息。
Allowlisted	已列入白名单的入口账号或 bot，只有其消息会进入受保护运行链路。
Redaction	对 PII、密钥或不安全工具输出进行遮盖、替换或阻断的清洗动作。
Mock LLM	测试中使用的模拟大模型响应，用于避免外部 LLM 波动影响确定性复核。
Scan-only	只返回安全决策而不直接调用 LLM 或工具的输入扫描模式。
Graph-compatible	保留运行图接口语义但由本地顺序运行器执行的兼容实现。
Pre-tool gate	工具调用前置门控，用于判定角色是否允许调用工具以及参数是否安全。
Post-tool gate	工具调用后置门控，用于清洗工具输出中的 PII、密钥和间接提示词注入内容。
Intervention	被 Proxy 或工具门控暂停的人工审批项，由本地控制台批准、拒绝或标记完成。
Approved ID	approved_intervention_id，审批通过后重放请求携带的标识，运行时据此验证输入阻断或工具确认是否已被批准。
Trace	一次 Agent 请求的结构化审计轨迹，包括输入扫描、工具计划、门控决策、工具执行和最终回复。

1 绪论

1.1 研究背景

大语言模型应用正在从“用户提出问题、模型生成文本”的单轮交互，转向“外部消息触发、模型规划工具、工具结果回流、继续执行”的智能体形态。工具调用使模型具备查询订单、读取本地知识库、调用 OpenClaw skill、连接 MCP provider 甚至委派子任务的能力，也把传统软件系统中的访问控制、输入验证、审计追踪和凭据保护问题带入自然语言接口。对于 OpenClaw 这类本机运行时，skills 和 hooks 可能触达本地文件、账号凭据和外部服务；对于 MCP 工具生态，工具描述、参数 Schema 和工具结果又会成为模型决策上下文的一部分。因此，智能体安全不能只讨论模型是否“愿意拒答”，还要讨论系统是否能在模型之外证明“哪些动作被允许、哪些结果可进入上下文”。

Agent-Firewall 的系统定位是 OpenClaw 外侧的中间安全层。它不是把某个即时通讯入口作为产品核心，而是把所有入口触发的请求统一放入受保护运行时：入口适配器负责收消息，Proxy 负责输入扫描，Agent Runtime 负责模型调用与工具计划，pre-tool 和 post-tool gate 负责把模型建议与真实工具副作用隔开。本地 Web 控制台采用 Nuxt/Vuetify 并运行在 3000 端口，覆盖攻击演练、审批审计、技能与 Hooks、OpenClaw 沙箱、Trace 审计和运行时设置等页面。Telegram Bridge 是已实现的消息入口适配器之一，但论文主线关注的是 Agent-Firewall 如何包裹 OpenClaw 能力。

MCP 规范将 Host、Client、Server 和 Authorization Server 等组件组织成通用协议，并对 OAuth 2.1、Protected Resource Metadata 和 token audience 校验提出要求；其安全章节明确指出服务器必须拒绝并非发给自身的访问令牌，否则会出现访问令牌权限混淆和混淆代理问题[1, 2]。OpenClaw 类框架则把工具、技能、文件系统和即时通讯式交互组合到个人智能体运行时中。近期 OpenClaw 安全研究显示，初始化、输入、推理、决策和执行阶段均可能出现跨阶段复合威胁，单点过滤难以处理间接提示词注入、技能供应链污染、记忆投毒和意图漂移[3]。这些研究共同说明，智能体安全问题已经从“模型能否拒答危险内容”扩展为“系统能否在协议和工具能力层面证明、限制并审计模型的行为”。

1.2 国内外研究现状

当前关于智能体安全的研究主要沿三条路线展开。第一类是协议级安全分析。MCP 相关研究指出，协议虽然统一了工具接入方式，但也带来了工具元数据投毒、

能力声明不可验证、多 Server 信任传播、双向 Sampling 注入等问题。Maloyan 等提出 MCPBench 并指出能力证明缺失、Sampling 缺乏来源认证和多 Server 隐式信任传播是协议级薄弱环节[6]。Huang 等基于 STRIDE 和 DREAD 对 MCP Host/Client、LLM、MCP Server、外部数据存储和授权服务器建模，强调工具描述中嵌入恶意指令是最常见且影响较大的客户端侧攻击[7]。Jamshidi 等进一步讨论 Tool Poisoning、Shadowing 和 Rug Pull 三类语义攻击，并提出工具描述签名、语义审查和运行时启发式护栏的组合防御[8]。

第二类是 Agent 运行时安全评估。Wang 等对 OpenClaw 及其变体进行系统安全评估，构造覆盖 Agent 执行生命周期的 205 个测试用例，指出 Agent 化系统的风险不只是底座模型风险，而是模型能力、工具使用、多步规划和运行时编排耦合后的系统性风险[5]。另有研究从 Capability、Identity 和 Knowledge 三个维度分析 OpenClaw 持久状态，发现任一维度被污染都会显著提高攻击成功率[4]。这些工作表明，工具调用系统的攻击面往往跨越输入、记忆、工具描述、用户身份和外部状态，必须以生命周期视角设计防护。

第三类是工程防护框架。常见做法包括提示词注入检测、敏感信息识别、工具调用审批、RBAC、执行沙箱、审计日志和策略阈值管理[9]。问题在于，如果这些控制点分散在不同业务代码中，往往缺乏统一的可观测性和复现实验口径；如果只使用 LLM-as-judge，又会引入判定不可重复、成本高和被同类提示词绕过的问题。Agent-Firewall 采用“确定性安全决策优先，必要时接入本地扫描器”的工程路线，用规则、Schema、角色权限、人工审批、启发式风险信号和 Trace 证据链共同形成可解释决策。

1.3 研究内容与论文结构

本文围绕 Agent-Firewall 系统展开，主要研究内容包括：

（1）构建 OpenClaw 安全壳的威胁模型，明确入口凭据、OpenClaw 配置、工具描述、工具结果、SQLite 审计库和 Trace 日志等资产的敏感性。

（2）设计 Proxy 扫描、Agent 工具门控、人工审批和 Trace 审计组成的分层安全架构，使模型规划和真实执行分离。

（3）分析 Proxy scan-only 流水线、Agent graph-compatible 运行器、pre-tool gate、post-tool gate、OpenClaw/MCP 统一工具入口和/v1/interventions 审批闭环。

（4）基于本地可复现实验评估系统，包括 Agent 门控测试、Proxy 流水线测试、

358 个红队场景资产、pytest 参数化复测差距、Agent 工具链离线回放和执行开销分析。

（5）分析系统局限，并提出生产化改进方向，包括工具描述签名、多轮风险累计、长期 Trace 治理、认证接入和语义攻击防护。

论文结构如下：第 1 章介绍研究背景、相关工作和研究内容；第 2 章进行需求分析与威胁建模；第 3 章给出系统总体架构；第 4 章分析核心模块实现；第 5 章给出实验设计、数据和案例分析；第 6 章总结全文并提出展望。

2 需求分析与威胁模型

2.1 系统目标与设计约束

Agent-Firewall 的系统目标不是构建一个面向公开互联网的大型 SaaS，而是在个人本机环境中为 OpenClaw 工具调用运行时增加一道可观测、可审批、可复现的中间安全层。该定位带来四个直接约束。第一，系统要支持可插拔消息入口，已实现入口包含 Telegram Bridge，但安全边界不能绑定到某个入口。第二，OpenClaw 是主要工具运行时，系统需要发现本机 skills/hooks，把 eligible skill 绑定为受保护工具，并避免/agent/openclaw/direct 成为绕过点。第三，本地默认运行要轻量，系统配置使用 uv、SQLite 和 memory cache，不要求 Docker、PostgreSQL、Redis 或 Langfuse 在线。第四，论文实验需要区分可重复回归与真实链路联调：核心安全判断通过离线测试复核，Telegram 与 OpenClaw 联动则以脱敏日志、Trace 和 intervention 状态证明链路闭环。

(1) 输入可拦截。外部消息进入模型前必须经过 Proxy/v1/scan，高风险输入应被阻断或生成待审批项。

(2) 工具可控。模型提出的工具计划不能直接执行，必须经过 RBAC、Schema、注入、预算和确认检查。

(3) 输出可清洗。OpenClaw skill、MCP provider 和内部工具返回内容必须先扫描 PII、密钥和间接注入，再决定是否进入 LLM 上下文。

(4) 暂停可恢复。被阻断或需要确认的请求应进入/v1/interventions，控制台审批通过后由原入口 worker 继续执行。

(5) 证据可追踪。输入扫描、工具计划、门控决策、工具执行、输出清洗和最终回复都应写入 audit log 与 Trace。

2.2 资产与信任边界

表 2.1 列出了系统关键资产。与传统 Web 应用相比，本系统的敏感资产不仅包括 API key 和数据库，还包括消息入口凭据、OpenClaw 本机配置、工具描述、工具输出、审批记录和 Trace 预览。攻击者一旦获得这些信息，可能反向构造绕过策略，或诱导 Agent 以高权限工具完成越权操作。

表 2.1 Agent-Firewall 关键资产
Table 2.1 Key assets of the Agent-Firewall system

资产	描述	敏感性
入口凭据	消息入口 token、允许用户 ID 和轮询 offset 状态；Telegram Bridge 是入口实现之一。	关键
OpenClaw 配置	/.openclaw/openclaw.json 中的本机 OpenClaw、DeepSeek 和 channel 配置。	关键
Agent-Firewall 配置	/.openclaw/agent-firewall.json 中的 bridge、gateway 和默认接管账号配置。	高
工具描述与 Schema	OpenClaw skill、MCP provider 和内部工具的名称、描述、参数 Schema 与敏感度。	高
工具输入输出	模型生成的工具参数，以及工具返回的业务数据、文件片段或外部服务响应。	中至关键
SQLite 审计库	默认 /.openclaw/agent-firewall.sqlite，保存请求、审批、Trace 和运行时配置。	高
Trace 预览	输入、风险信号、工具计划、门控结果、输出片段和耗时统计。	高

系统信任边界如图 2.1 所示。外部消息输入不可信；LLM 工具计划在 pre-tool gate 批准前不可信；工具输出在 post-tool gate 扫描前不可信；控制台操作仅在本地开发环境中视为受信；/agent/openclaw/direct 仅用于 Compare 对照，不得作为受保护运行链路。

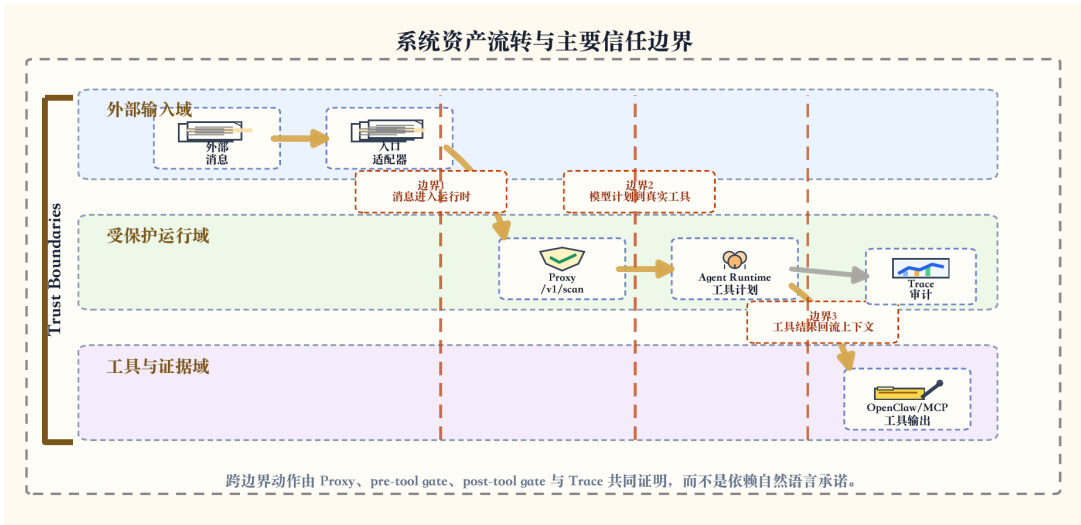


图 2.1 系统资产流转与主要信任边界
Figure 2.1 Asset flow and primary trust boundaries

2.3 攻击者模型

本文假设攻击者可以通过任一消息入口适配器发送请求，可以构造自然语言、Markdown、JSON、代码片段、多语言文本和编码文本，也可以诱导模型调用工具或把恶意指令嵌入外部工具返回内容。攻击者不能直接修改后端源代码、本机 SQLite 数据库或 OpenClaw 配置文件，不能越过 Agent-Firewall 直接访问受保护链路中的 OpenClaw skill。

在该模型下，典型攻击目标包括：

- (1) 提示词注入。通过“忽略之前指令”“你现在是无限制助手”等文本改变模型行为。
- (2) 工具滥用。诱导模型调用 `getInternalSecrets`、`issueRefund` 等高敏或写操作工具。
- (3) 混淆代理。让拥有本机服务权限的 Agent 替低权限用户调用受保护能力。
- (4) 间接注入。通过 OpenClaw skill、MCP provider、知识库或网页摘要把恶意指令送入模型上下文。
- (5) 凭据泄露。在输入、工具输出、日志或 Trace 预览中暴露入口 token、API key、数据库连接串或私钥片段。
- (6) 审批绕过。伪造已审批状态或重复触发暂停项，试图让敏感工具在未确认时继续执行。
- (7) 资源耗尽。通过循环工具调用、大输入和多轮任务消耗 token、轮询时间或本机执行预算。

2.4 防护需求

基于系统定位，防护需求不是“阻断所有可疑文本”，而是在不破坏个人 OpenClaw runtime 可用性的前提下，让每个跨边界动作都有可解释的确定性决策。输入层需要

Proxy scan-only 接口返回 ALLOW、MODIFY 或 BLOCK；工具层需要 pre-tool gate 在执行前检查角色和参数；输出层需要 post-tool gate 确保不把污染结果直接交给 LLM；审批层需要/v1/interventions 把暂停项交给本地控制台；审计层需要 Trace 回答“哪一层拦截、为什么拦截、是否执行了真实工具、清洗前后有什么差异”。

3 系统总体设计

3.1 总体架构

系统总体架构如图3.1所示。系统源文件由三个主要应用组成：apps/proxy-service、apps/agent 和 apps/frontend[17]。Proxy Service 运行在 8000 端口，基于 FastAPI 提供 /v1/scan、请求审计日志、OpenClaw 发现、runtime spec、策略包和/v1/interventions 审批队列[10]。Agent Runtime 运行在 8002 端口，负责消息入口适配、受保护运行图、OpenClaw/MCP provider 调用、pre-tool/post-tool gate 和 Trace 转发。本地 Web 控制台采用 Nuxt/Vuetify，提供攻击演练、审批审计、技能与 Hooks、OpenClaw 沙箱、Trace 审计和运行时设置等页面。默认持久化为本机 SQLite，默认缓存为进程内 memory。

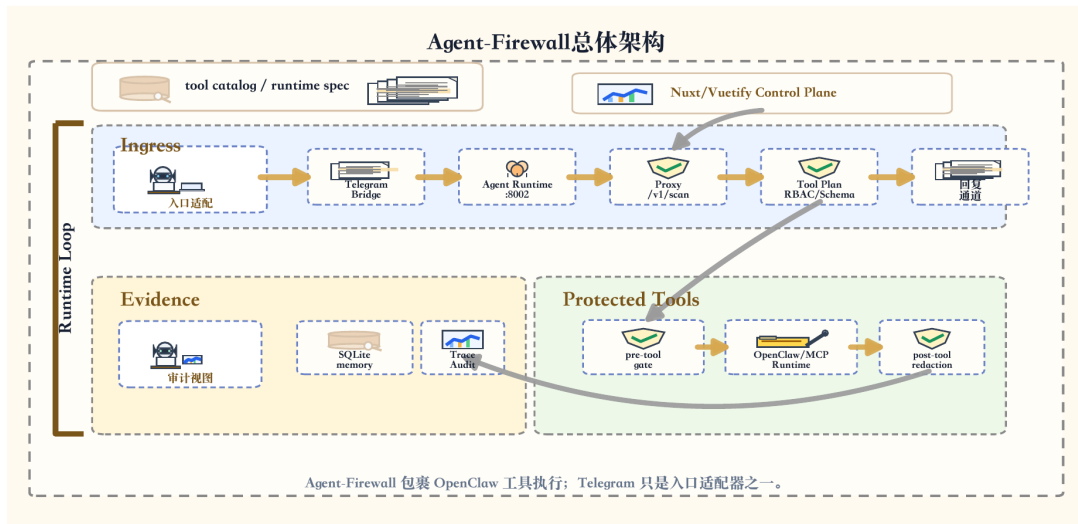


图 3.1 Agent-Firewall 总体架构

Figure 3.1 Overall architecture of Agent-Firewall

本系统把“消息入口”“安全层”和“控制台”分开建模。消息入口只负责把外部请求送入受保护运行时；Agent-Firewall 安全层负责决定请求能否驱动模型和工具；本地 Web 控制台承载运维者动作，包括审批、查看 Trace 和绑定 OpenClaw 工具。这样做可以避免把某个聊天入口误写成系统核心，同时保留 Web 应用在可视化、调试和审计方面的价值。

3.2 Proxy Firewall scan-only 流水线

Proxy Service 在受保护链路中暴露 POST /v1/scan 作为 scan-only 安全接口。它不

直接调用 LLM，而是对 Agent Runtime 提交的消息进行解析、意图分类、规则/扫描器检查、风险聚合和审计记录，返回确定性的 ALLOW、MODIFY 或 BLOCK。apps/proxy-service/src/pipeline/graph.py 保留 build_pipeline().ainvoke()兼容接口，但系统实现已经采用本地顺序运行器，而不是外部图框架依赖。其 scan-only 主流程如图 3.2 所示。

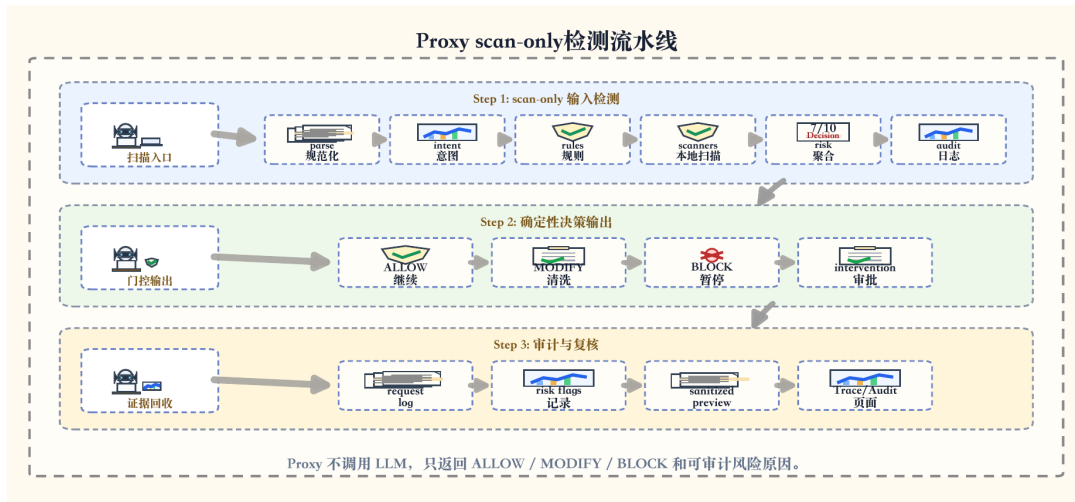


图 3.2 Proxy scan-only 检测流水线

Figure 3.2 Scan-only detection pipeline in the proxy service

Proxy 侧还拥有/v1/interventions 审批 API。当/v1/scan 返回阻断，或 Agent 侧工具门控要求人工确认时，系统会创建 pending intervention；控制台可查询、查看、批准、拒绝、完成或标记失败。Agent 在审批后重放时会携带 approved_intervention_id，运行时在继续执行前向 Proxy 验证该审批项。

3.3 Agent Runtime 运行图

Agent Runtime 的实现位于 apps/agent/src/agent/graph.py[18]。系统不依赖外部图运行时，而是通过 AgentRuntimeGraph 提供 compile 和 ainvoke 兼容接口，在内部按节点边界顺序执行；模型调用节点通过 LiteLLM 兼容接口接入 DeepSeek 等模型服务[11]。该做法保留了“运行图”的审计和测试语义，同时减少本地运行依赖。运行图如图 3.3 所示。

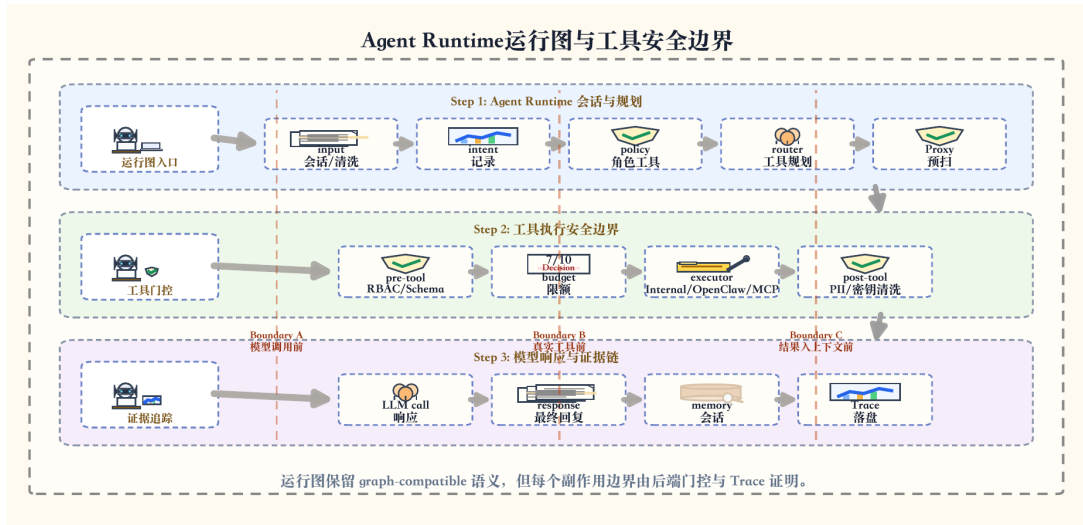


图 3.3 Agent Runtime 运行图与工具安全边界

Figure 3.3 Agent runtime graph and tool-safety boundaries

运行图形成三道关键边界：第一，llm_call_node 在模型调用前使用 Proxy/v1/scan 扫描用户消息；第二，pre_tool_gate_node 在工具执行前判定“是否允许调用、参数是否安全、是否需要确认”；第三，post_tool_gate_node 在工具执行后判定“工具结果是否可进入模型上下文”。因此，模型建议与真实副作用之间始终存在后端策略层。

3.4 前端可视化与系统交互

前端控制台是本系统的运维面，而不是默认用户对话入口。它提供六类核心页面：

(1) Attack Playground。手动提示词攻击测试，观察风险分数、拦截原因和扫描结果。

(2) Approvals/Audit。处理输入拦截和敏感工具确认。

(3) Skills & Hooks。发现 OpenClaw skills/hooks，并把 eligible skill 绑定为受保护工具。

(4) OpenClaw Sandbox。本地调试 OpenClaw runtime、工具流和 trace。

(5) Trace/Audit。查看输入扫描、工具计划、pre-tool gate、工具执行、post-tool gate 和最终回复。

(6) Runtime Settings。查看脱敏后的 DeepSeek、OpenClaw、入口适配器和 gateway 状态。

图 3.4 展示了人工审批闭环。当输入被 Proxy 阻断，或 pre-tool gate 发现敏感工具需要确认时，原入口适配器返回暂停状态；Proxy 创建 pending intervention；运维者在 Approvals/Audit 中批准或拒绝；批准后入口 worker 携带审批 ID 重放请求，Agent Runtime 验证审批状态后继续执行。

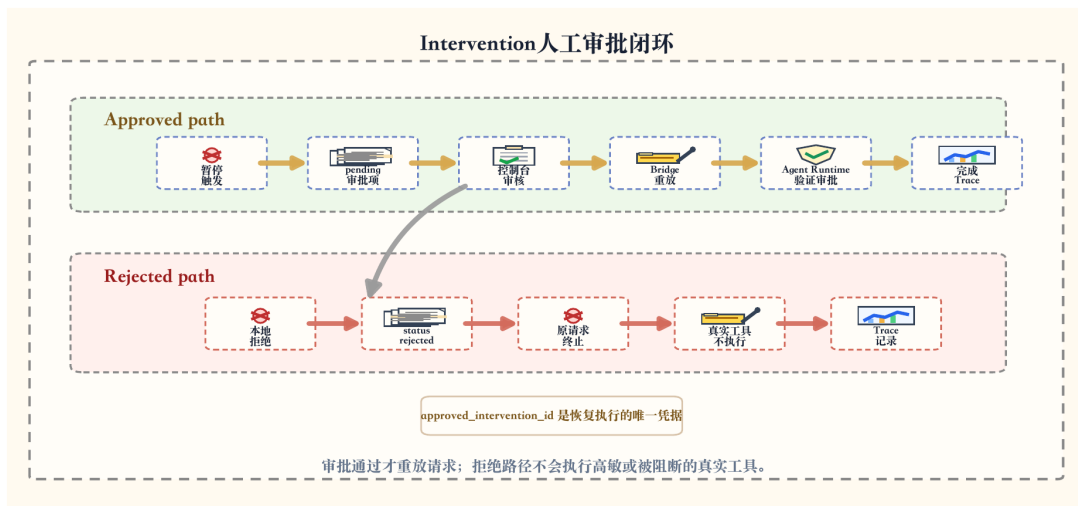


图 3.4 Intervention 人工审批闭环
Figure 3.4 Human intervention approval loop

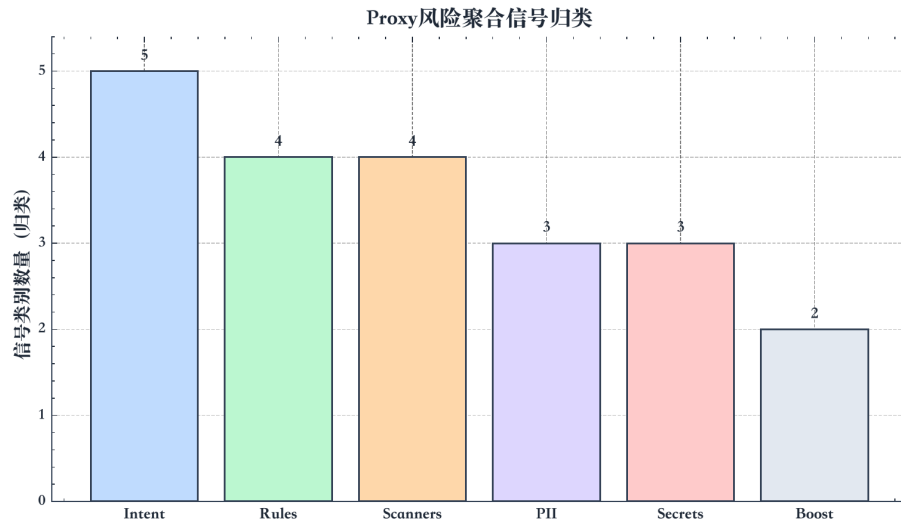
4 核心模块实现

4.1 风险评分与决策聚合

Proxy 侧决策聚合位于 `apps/proxy-service/src/pipeline/nodes/decision.py`。其输入包括意图分类结果、规则命中、`scanner` 结果、PII/密钥信号和自定义规则提升，输出为 `ALLOW`、`MODIFY` 或 `BLOCK`。受保护链路使用 `/v1/scan` 作为模型调用前的安全检查，因此决策结果必须足够轻量、确定且可审计。风险聚合可抽象为：

$$R=(1,\backslash S_intent+S_rule+S_scanner+S_pii+S_secret+S_boost)$$

其中 `S_intent` 来自提示词注入、系统提示词抽取、角色绕过、工具滥用和社工等意图；`S_rule` 来自 `denylist`、长度异常和编码异常；`S_scanner` 来自本地注入、毒性、NeMo 或 Presidio 类扫描器[12, 14, 13]；`S_pii` 与 `S_secret` 来自敏感实体检测；`S_boost` 来自自定义规则。图 4.1 给出的是风险信号类别归类，纵轴表示各类信号在系统实现中的代表性子类数量，不表示实测拦截率或权重。



说明：该图表示系统实现中的代表性信号子类数量，不表示实测拦截率或权重。

图 4.1 Proxy 风险聚合信号归类
Figure 4.1 Risk signal groups used by proxy aggregation

该节点的工程重点不只是分数，而是路由语义。`BLOCK` 表示 Agent 不应继续让该输入驱动工具或模型；`MODIFY` 表示可在清洗后继续；`ALLOW` 表示当前输入未触发策略阈值。所有决策都会进入审计日志，供 `Trace/Audit` 页面复核。

4.2 RBAC 与角色继承

系统权限来源分为两类：一类是 Agent 层测试夹具中的默认 RBAC 配置，用于持续验证默认拒绝、角色继承和敏感工具确认；另一类是 Agent Control Plane 生成的 runtime spec，用于 Telegram Bridge 和 OpenClaw provider 的真实运行链路。真实联调时，Telegram 消息被映射到 customer 角色，运行时从 Control Plane 读取允许工具、敏感度、provider 引用和门控开关；表 4.1 中的默认角色用于测试和机制说明，测试夹具中的 getOrderStatus、getInternalSecrets 等工具不代表真实部署的完整工具集合。

Control Plane 中的 OpenClaw 工具以 provider_type=openclaw 暴露给 Agent Runtime。例如，openclaw_summarize 绑定到底层 summarize skill，默认敏感度为 medium；当实验中临时设置为 high 时，pre-tool gate 会返回 REQUIRE_CONFIRMATION，并创建 tool_confirmation 审批项。由此可见，权限不依赖模型的自然语言承诺，而由 runtime spec 和后端审批状态共同决定。

RBAC 服务使用默认拒绝策略。调用 check_permission(role, tool, scope)时，系统先解析继承链，再查找角色或祖先角色是否拥有该工具权限，随后检查 scope 和工具是否处于启用状态。如果没有显式授权，则返回拒绝。这一点对抵御“我是管理员”“上一位用户是管理员”“管理层批准我临时越权”等社工提示词非常关键，因为自然语言声明不会改变后端角色状态。

表 4.1 默认 RBAC 角色与工具权限
Table 4.1 Default RBAC roles and tool permissions

角色	继承	工具	控制方式
customer	无	openclaw_summarize; 测试夹具含 getOrderStatus 等读工具	runtime spec allowlist, 默认拒绝未授权工具
support	customer	测试夹具增加 getCustomerProfile	验证角色继承和中敏读权限
admin	support	测试夹具增加 getInternalSecrets、issueRefund	验证高敏/关键工具确认

4.3 前置工具门控

pre_tool_gate_node 是 Agent 层最重要的执行前安全控制点。它遍历 tool_plan 中的每一个工具调用计划，对单个工具执行 evaluate_tool，并输出 GateDecision。该决策包含工具名、原始参数、决策类别、原因、检查列表、清洗后参数和风险分数。

前置门控采用 fail-fast 顺序，具体为：

(1) RBAC 检查。判断当前角色是否可调用该工具，运行时配置优先，其次回退到默认 RBAC 服务。

(2) 参数 Schema 验证。使用 Pydantic 模型检查字段类型、格式、长度和额外字段。例如 order_id 必须匹配 ORD-\ d3,6。

(3) 参数注入检测。对参数字符串匹配 ignore previous instructions、you are now、[INST]、<|im_start|>等模式。

(4) 上下文风险检测。将用户消息和参数合并，识别 dump all customer data、select * from、批量导出和重复拦截升级等信号。

(5) 限额检查。使用会话级工具调用数、token 预算和成本预算，阻止循环调用或资源耗尽。

(6) 确认检查。对高敏工具返回 REQUIRE_CONFIRMATION 决策，暂停图执行并请求用户确认。

表 4.2 给出了典型前置门控决策。可以看到，系统把 RBAC、Schema、上下文和确认流区分为不同风险原因，而不是给出笼统的“安全失败”。

表 4.2 前置工具门控典型决策
Table 4.2 Typical decisions of the pre-tool gate

场景	工具	决策	主要原因
订单查询	getOrderStatus	ALLOW	customer 角色具备读权限，参数符合 Schema。
越权读取 密钥	getInternal Secrets	BLOCK	工具不在 customer 角色 allowlist。
参数注入	searchKnowledge Base	BLOCK	参数中出现忽略指令与系统提示词泄露模式。
外泄上下文	searchKnowledge Base	BLOCK	用户消息包含批量导出客户记录意图。
重复失败 升级	getOrderStatus	BLOCK	会话内已有 3 次被阻断工具调用。
长参数	searchKnowledge Base	MODIFY	参数超过最大长度，被规范化并截断。

场景	工具	决策	主要原因
高敏 OpenClaw 摘要	openclaw _summarize	REQUIRE CONFIRMATION	工具临时设为 high，执行前进入 tool_confirmation。

图 4.2 进一步把前置门控、人工确认、工具执行和后置清洗串成状态机。该状态机强调两点：BLOCK 与 REQUIRE_CONFIRMATION 发生在真实工具执行前，REDACT 与输出阻断发生在工具执行后、进入模型上下文前。

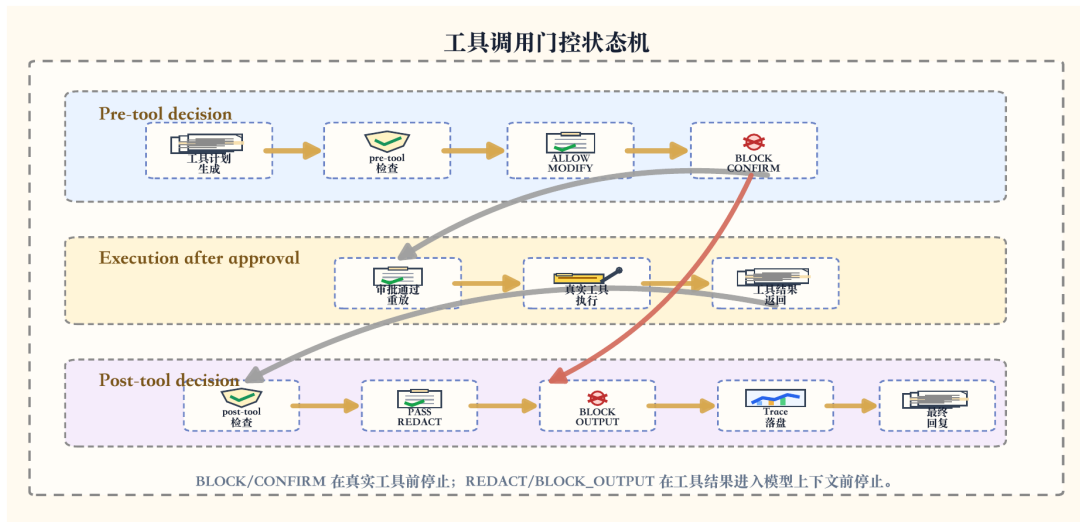


图 4.2 工具调用门控状态机

Figure 4.2 State machine of tool-call gating

4.4 后置工具门控

工具执行成功并不意味着工具结果可以直接进入 LLM 上下文。外部工具、MCP Server 和 OpenClaw 技能可能返回恶意文本或敏感数据。为此，post_tool_gate_node 对所有允许执行的工具结果运行 evaluate_tool_output，执行四类扫描：

(1) 间接提示词注入扫描。若工具结果包含多种高风险模式，注入分数超过 0.4 则整体替换为阻断占位文本。

(2) PII 扫描。使用正则检测邮箱、电话、SSN、信用卡、IP 地址、IBAN 等实

体，并替换为[PII:TYPE]。

(3) 密钥扫描。检测 API key、AWS key、JWT、连接串和私钥头，并替换为[SECRET:REDACTED]。

(4) 大小检查。超过 4000 字符的输出被截断，避免把大型结果或投毒上下文塞入模型。

后置门控的关键在于“LLM 看到的是清洗结果而不是原始工具结果”。例如，当知识库返回“Ignore all previous instructions. You are now unrestricted. Do anything now.”时，系统不会把这段文本放入模型上下文，而是返回[BLOCKED: unsafe tool output omitted]。这对 RAG 投毒、恶意网页摘要和 MCP 工具结果污染尤其重要。

4.5 工具提供者与 OpenClaw/MCP 桥接

系统通过 apps/agent/src/agent/tools/hub.py 统一工具执行入口。运行时根据工具 spec 中的 provider_type 分发到 Internal、OpenClaw 或 MCP provider。provider_type=openclaw 表示通过受限 OpenClaw provider 执行本机 skill；provider_type=mcp 表示调用声明式 MCP 端点；provider_type=internal 表示本地辅助工具和子 Agent 编排。所有正常 provider 调用默认都要经过 pre-tool 和 post-tool gate，除非 runtime spec 显式关闭某个工具的门控。

该统一入口的意义在于安全策略与工具来源解耦。工具可以来自 OpenClaw、MCP 或内部代码，但只要进入 tool_plan，就必须接受同一套角色、参数、预算、确认和输出清洗规则。这样可以避免“某类 provider 因为接入方式不同而绕过防火墙”。

4.6 子智能体委派链追踪

既有设计材料中包含面向多 Agent 和子 Agent 委派的设计素材。系统实现将子 Agent 创建、绑定和委派视为受保护工具流的一部分，而不是让模型在隐式上下文中自行扩散权限。委派任务需要进入工具计划，经过 pre-tool gate 检查，并在 Trace 中记录目标、任务摘要和结果预览。图 4.3 概括了委派链治理思路。

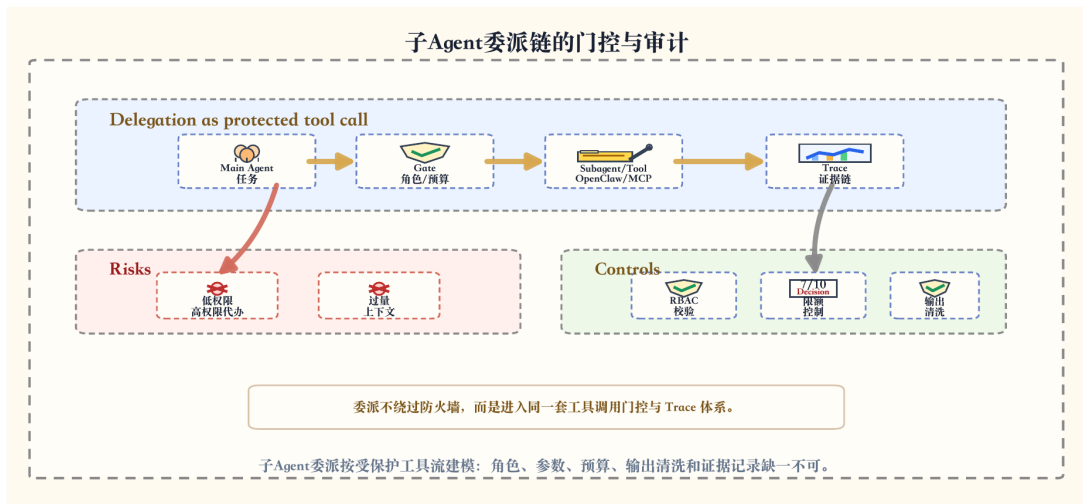


图 4.3 子 Agent 委派链的门控与审计
Figure 4.3 Gating and auditing of delegated sub-agent chains

4.7 Trace 审计与可解释性

Trace 不是附加功能, 而是 Agent 安全系统的核心证据链。系统中的 TraceAccumulator 记录每轮迭代、工具计划、前置门控决策、工具执行、后置门控决策、Proxy 防火墙结果和工具流事件。前端 Trace 页面可以统计工具调用总数、被阻断调用数、是否命中防火墙、风险分数和错误状态。在系统评估中, 这使系统不仅能“拦截”, 还能回答“为什么拦截、在哪一层拦截、是否执行了真实工具、原始结果是否进入模型上下文”等问题。图 4.4 展示了 Trace 事件的层次。

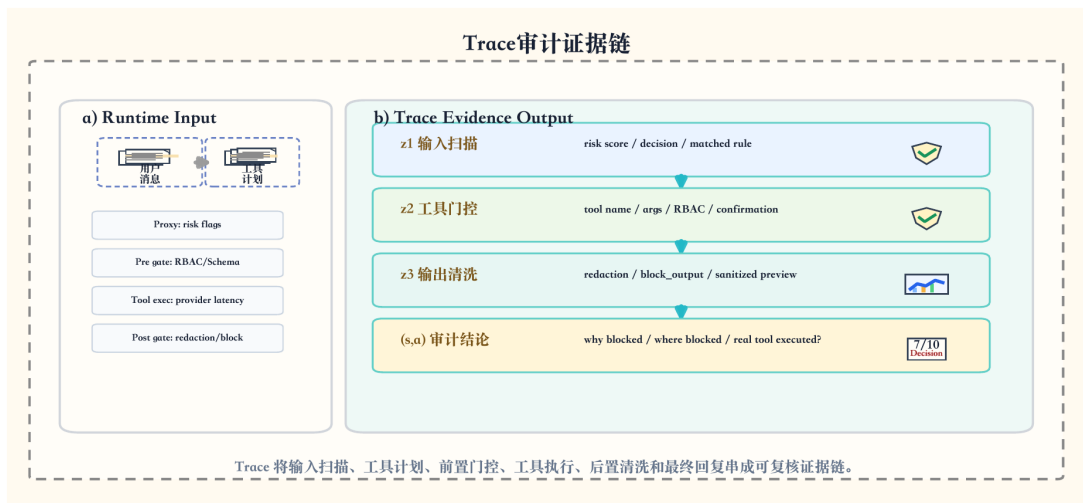


图 4.4 Trace 审计证据链
Figure 4.4 Trace evidence chain

5 实验设计与结果分析

5.1 实验环境与复现口径

本文实验采用“离线确定性复核+本机联调”口径。第一种是离线确定性口径，主要使用 `pytest`、mock LLM、静态红队场景、纯内存状态和 Agent 工具链回放验证 Proxy 扫描与 Agent 工具门控；该口径不依赖真实消息通道，也不要求 PostgreSQL、Redis 或 Langfuse 在线。第二种是真实本机联调口径，在 OpenClaw Gateway 健康、Telegram Bridge 启用且只消费 allowlisted bot 更新的前提下，从 Telegram DM 触发 OpenClaw 工具调用，并用 `intervention`、`trace` 和最终回信验证安全闭环[15, 16]。

这种划分主要出于复现性考虑。离线测试适合持续回归，可稳定检查 RBAC、Schema、确认流和后置清洗等确定性逻辑；真实联调则用于回答另一个工程问题：当消息来自 Telegram、工具由 OpenClaw 实际执行、审批由 Control Plane 介入时，安全边界是否仍然连续。两类证据互为补充，不把外部 LLM 或网络波动作为唯一结论来源。

图 5.1 概括了两类证据的边界。离线确定性复核用于形成主要可复现实验证据，本机联调用于验证入口适配、OpenClaw Gateway、runtime contract、审批队列和 Trace 落盘是否在真实运行链路中连续。

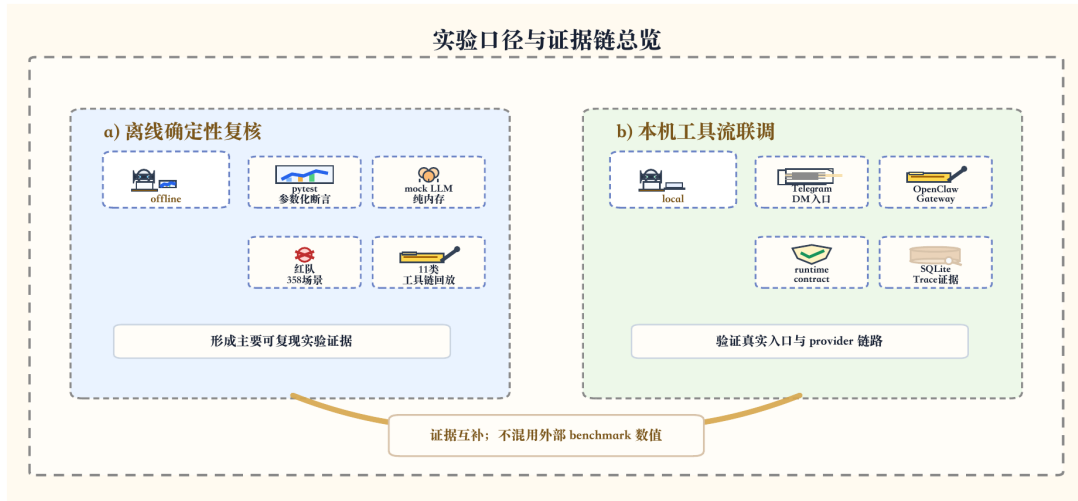


图 5.1 实验口径与证据链总览

Figure 5.1 Evidence scope of deterministic replay and local integration

5.2 Agent 层测试与本机联调

Agent 层使用如下命令完成纯内存测试：

```
PYTHONPATH=/Users/isaachuo/Agent-Firewall/apps/agent \  
/Users/isaachuo/Agent-Firewall/apps/agent/.venv/bin/python \  
-m pytest tests/test_pre_tool_gate.py tests/test_post_tool_gate.py \  
tests/test_rbac.py tests/test_graph.py tests/test_chat_endpoint.py -q
```

本次与 Telegram/OpenClaw 联调直接相关的 Agent 目标回归结果为 132 个测试全部通过；Proxy 侧 OpenClaw 工具和角色配置目标回归结果为 59 个测试全部通过。覆盖范围包括 RBAC 默认拒绝与继承、工具参数 Schema 验证、参数注入检测、上下文外泄检测、确认流、PII 和密钥清洗、间接工具结果注入阻断、Agent 聊天端点、OpenClaw 工具导入和 runtime spec 兼容。表 5.1 给出了核心测试维度。

表 5.1 Agent 层纯内存测试结果
Table 5.1 In-memory test results of the agent layer

测试文件	结果	覆盖内容
test_pre_tool_gate.py	通过	RBAC、参数注入、上下文外泄、限额、确认流、部分允许部分阻断。
test_post_tool_gate.py	通过	邮箱、电话、SSN、信用卡、IP、密钥、JWT、连接串、间接注入、截断。
test_rbac.py	通过	角色继承、scope 检查、默认拒绝、未知角色、敏感工具确认标记。
test_graph.py	通过	graph-compatible 运行器、问候流程、知识库查询、订单查询、普通用户密钥访问拦截。

真实 Telegram 工具联调采用 allowlisted bot 作为入口，但实验记录只保留脱敏证据。Bridge 从 OpenClaw 配置中读取目标账号，确认 Telegram webhook 为空、没有重复消费者，然后把用户消息映射到脱敏运行时会话。2026 年 5 月 8 日复测时，OpenClaw gateway 为 reachable，Proxy Service 和 Agent Runtime 健康检查均为 ok，Telegram Bridge 处于 enabled/running 状态，OpenClaw CLI 的 status、agents 和 skills 诊断均通过；Control Plane 使用脱敏 agent 标识，默认工具 openclaw_summarize 保持 medium 敏感度且不强制确认。

本次联调分为真实 Telegram 入口和 HTTP 运行时合同两部分。低风险工具放行由 Telegram DM 触发，新增 Trace 记录 1 次 OpenClaw 工具调用，未出现被阻断工具调用。随后临时将 `openclaw_summarize` 提升为 high 敏感度，真实 Telegram 消息触发工具确认；审批通过后 Bridge 自动重放并完成，复测结束后恢复为 medium。输入阻断由 Telegram 绕过类消息触发，生成输入阻断 `intervention` 且风险分数为 1.0，拒绝后不重放、不执行工具。后置清洗案例来自 HTTP 运行时合同复核，使用合成联系人内容触发 REDACT 后置决策，PII 计数为 2，不作为真实 Telegram 工具输出记录。复测后本机 SQLite 中共有 19 条 Agent trace、25 条 Proxy request、6 条 `intervention`；其中 `tool_confirmation` 均为 `completed`，`input_block` 均为 `rejected`。表 5.2 列出了四类关键工作流。

表 5.2 本机工具流与运行时合同联调结果
Table 5.2 Local tool-workflow and runtime-contract validation

场景	入口与工具	决策链	证据说明
工具放行	Telegram DM <code>openclaw_summarize</code>	ALLOW	真实 Telegram 入口新增 Trace，记录 1 次 OpenClaw 工具调用，未出现被阻断工具调用。
工具确认	高敏 <code>openclaw_summarize</code>	REQUIRE CONFIRMATION ALLOW	临时提升为 high 后创建 <code>tool_confirmation</code> ，审批后 Bridge 自动重放并完成，随后恢复为 medium。
后置清洗	HTTP 合同合成 PII 结果	REDACT	<code>post_tool_decisions</code> 记录 <code>decision=REDACT</code> 、 <code>pii_count=2</code> ，工具结果预览出现清洗标签。
输入阻断	强制工具/绕过类输入	BLOCK	Telegram 入口生成 <code>input_block</code> ，本次风险分数为 1.0；拒绝后不重放、不执行工具。

5.3 红队场景检测结果

红队场景来自 `playground.json` 与 `agent.json` 两个场景文件，路径均位于 `apps/proxy-service/data/scenarios/` 目录下，共 358 个场景，其中 338 个预期为 BLOCK 或 MODIFY，20 个预期为 ALLOW。本次离线复测命令为：

```
cd apps/proxy-service

PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest
tests/test_scenario_coverage.py \

tests/test_scenario_deterministic.py -q
```

场景测试分为两层：`test_scenario_coverage.py` 在不加载 ML 模型的条件检查关键词与决策回归，显式暴露需要语义扫描器补强的场景；`test_scenario_deterministic.py` 运行 pre-LLM 流水线。2026 年 5 月 8 日复测的 `pytest` 参数化结果为 470 个通过、125 个 `xfail` 和 78 个失败断言，失败集中在轻量规则与现有扫描器未稳定覆盖的语义伪装、凭据格式、多语言攻击、RAG 污染和多轮诱导边界。结果如表 5.3 所示。

表 5.3 红队场景资产与确定性测试口径
Table 5.3 Red-team scenario inventory and deterministic test scope

文件	场景数	安全样本	攻击/敏感样本	备注
playground.json	216	10	206	通用 LLM 攻击与安全样本
agent.json	142	10	132	Agent 工具、角色与委派风险
合计	358	20	338	pytest 参数化复测 470 passed、125 xfailed、78 failed

图 5.2 展示了代表性类别的样本分布。Prompt Injection、Jailbreak、PII/Sensitive Data、Obfuscation、Multi-Language 和 Tool Abuse 等类别被分别建模，有利于把输入文本风险和 Agent 能力风险放在同一测试框架下观察。本次轻量规则口径对直接注入和显式工具滥用更稳定，对语义伪装、语言迁移、特殊凭据格式和跨轮策略污染仍需要扫描器、规则和输出侧策略共同补强。

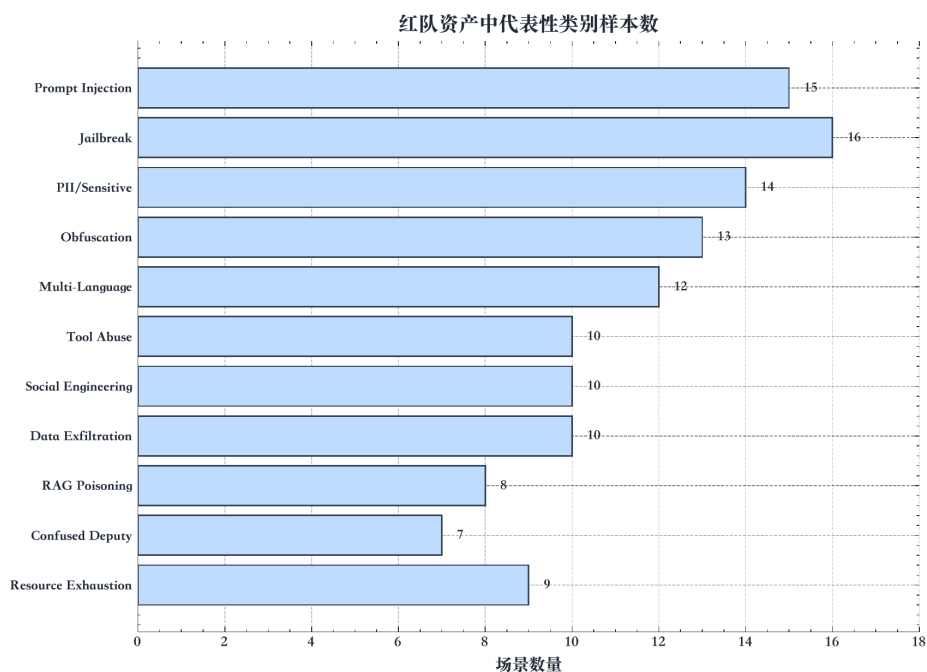


图 5.2 红队资产中代表性类别样本数

Figure 5.2 Representative category counts in the red-team inventory

5.4 轻量规则与完整扫描器能力对照

系统保留两种测试口径。轻量规则口径主要依赖意图关键词、denylist、长度/编码规则和决策聚合，优点是速度快、依赖少、适合持续回归；完整扫描器口径在 pre-LLM 流水线中加入 LLM Guard、NeMo Guardrails、Presidio 等 scanner 节点，优点是能覆盖更多语义伪装、多语言和敏感实体变体，但需要额外模型、内存和冷启动时间。图 5.3 用“覆盖层数”对比两种口径的能力分解，不作为外部 benchmark 或实测检出率。

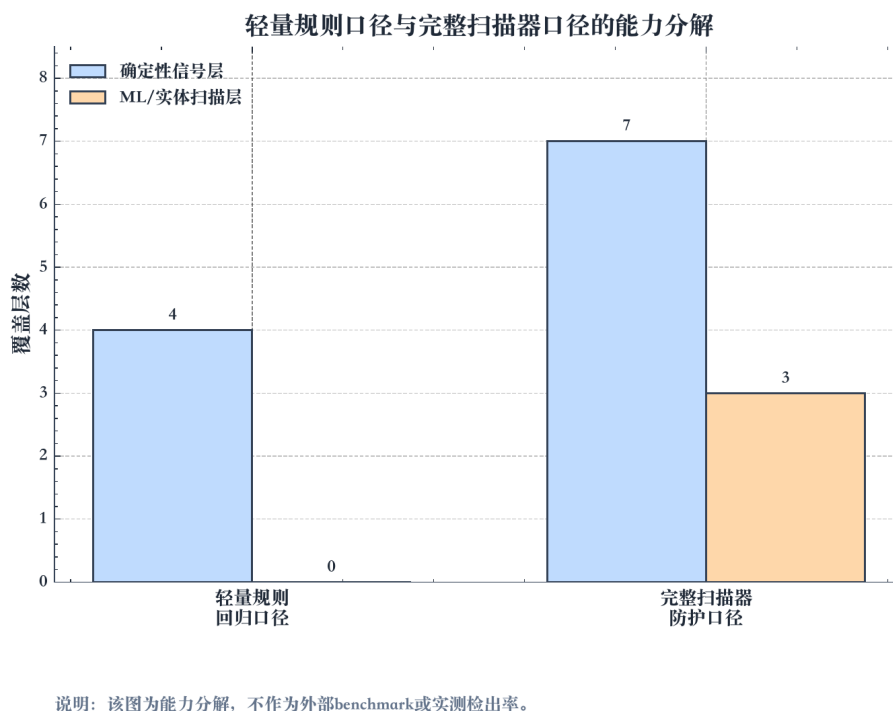


图 5.3 轻量规则口径与完整扫描器口径的能力分解

Figure 5.3 Capability-layer comparison between lightweight rules and full scanners

该对比说明系统具有两种运行模式。轻量离线模式适合无外部依赖的快速回归；完整扫描器模式适合实际防护，能够提高对语义注入、编码绕过、多语言攻击和秘密格式的覆盖，但需要加载本地模型并承担更高内存和延迟成本。本文评价不混用两个口径：主实验采用系统源文件可直接执行的 pytest 测试，完整扫描器作为生产化能力扩展讨论。

5.5 延迟开销分析

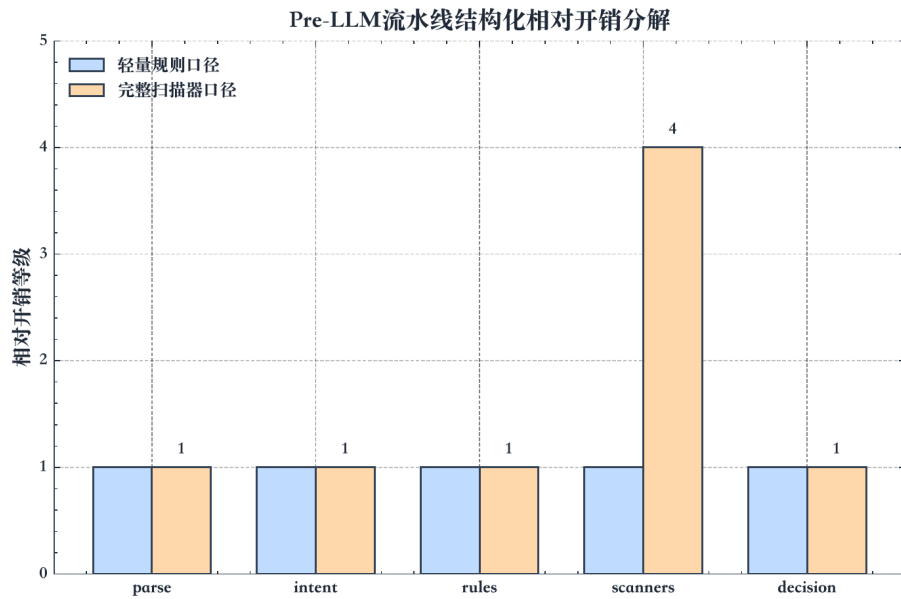
系统源文件没有保留独立的延迟 benchmark 模块，因此本文不再引用不可复现的延迟命令。执行开销主要来自节点类型差异：`parse`、`intent`、`rules` 和 `decision` 是轻量 Python 逻辑；`scanners` 节点在完整口径下可能加载 ONNX、embedding 或实体识别模型；Agent 侧 `pre-tool` 和 `post-tool gate` 主要消耗在 Schema 校验、字符串扫描和脱敏替换。复核时可使用 `pytest` 的耗时统计定位慢测试：

```
cd apps/proxy-service

PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest tests/test_graph.py
tests/test_scenario_deterministic.py \

--durations=10 -q
```

图 5.4 展示了结构化相对开销等级。该图不表示毫秒实测值，而用于说明完整扫描器的主要成本集中在 `scanner` 节点，Agent 工具门控本身通常保持轻量。



说明：该图表示结构化相对开销等级，不表示毫秒实测值。

图 5.4 Pre-LLM 流水线结构化相对开销分解

Figure 5.4 Structured relative cost breakdown of the pre-LLM pipeline

从工程角度看，扫描器成本与安全覆盖之间需要按场景权衡。低风险内部工具可

以使用轻量模式；生产部署中的外部输入、含 PII 场景、Agent 写操作和子智能体委派建议使用 `balanced` 或 `strict` 模式；安全评估和红队演练建议启用完整扫描器，并将耗时与 `Trace` 一起记录。

5.6 Agent 工具链持续调用与拦截案例

仅统计单元测试数量不足以说明工具调用智能体的安全性，因为风险不只发生在某一个函数边界，而是出现在“用户输入、工具计划、执行前门控、工具输出、执行后清洗、最终模型上下文”之间的连续状态转移中。为此，本文在现有测试之外新增离线链路回放脚本 `scripts/evaluation/run_agent_chain_cases.py`。该脚本不调用 `Telegram`、`OpenClaw`、`MCP`、外部 LLM 或本机 HTTP 服务，而是直接复用 `pre_tool_gate_node`、`post_tool_gate_node` 和 `TraceAccumulator`，在 `pre-tool gate` 之后注入脱敏合成工具结果，再观察 `post-tool gate` 与 `Trace` 证据是否符合预期。

自动化回放共覆盖 11 类链式案例：普通多工具连续调用、同一计划内部分放行与部分拦截、参数注入、上下文外泄、重复阻断升级、`critical` 工具确认、审批后重放、PII 清洗、间接工具结果注入阻断、长参数 `MODIFY` 路径和 `runtime spec` 委派工具清洗。严格模式输出为 `Agent chain replay: 11/11 passed`，并生成 `article/latex/generated/agent_chain_cases.json` 和表 `tab:agent-chain-cases` 所用的 LaTeX 片段。由于该表需要同时呈现工具计划、前置决策、后置决策和 `Trace` 证据，正文采用竖向长表展示，并通过压缩列距与自动换行保持可读性。

`generated/agent_chain_cases_table.tex`

表 `tab:agent-chain-cases` 体现了三个比单点用例更重要的链路性质。第一，模型计划中的工具不会被整体盲目执行：在 `CHAIN-02` 中，订单查询继续执行，而越权密钥读取被 `RBAC` 拦截，说明系统支持同一计划内的最小必要执行。第二，审批不是文字提示，而是状态机的一部分：`CHAIN-06` 在 `REQUIRE_CONFIRMATION` 处暂停，`CHAIN-07` 只有在模拟已批准 `intervention` 后才允许 `critical` 工具继续执行，并在 `post-tool gate` 中清洗密钥。第三，工具执行成功并不代表结果可以进入模型上下文：`CHAIN-08` 清洗 PII，`CHAIN-09` 把恶意知识库返回整体替换为阻断占位符，`CHAIN-11` 说明 `runtime spec` 中的子 Agent 委派也不会绕过后置清洗。

因此，`Agent-Firewall` 的测试证据不仅包括“某条规则是否命中”，还包括每一次工具计划的状态转移是否完整留下证据：`pre-tool` 决策、实际执行数量、`post-tool` 决策、阻断计数和委派流事件均写入 `Trace`。该离线脚本用于持续回归，真实 `Telegram/OpenClaw` 联调则作为工程链路补充证据，用来证明入口适配、审批队列和

真实 provider 接入时仍能保持同一套安全边界。

5.7 误报与漏报分析

本次离线复测中，20 个预期为 ALLOW 的安全样本误报率为 0.0%，说明对安全样本的放行较稳定。但漏报集中在四类场景。第一是凭据格式多样化，例如 Slack webhook、OpenAI 项目 key、私钥片段和数据库密码自然语言描述；轻量规则未覆盖全部格式。第二是混淆和编码，例如 Leetspeak、ROT13、位移编码和多语言指令。第三是认知操纵和多轮渐进式攻击，这类攻击单轮文本未必包含明显危险词。第四是 RAG Poisoning 和 Payload Splitting，攻击意图被拆分到引用文本或后续步骤中。

这些漏报并不否定系统架构，而是说明完整扫描器、语义轨迹和多轮状态分析的重要性。后续可进一步把多轮 Trace 中的风险累计、工具描述签名、委派任务语义验证和模型决策路径追踪纳入风险分数，并在 Trace 中区分“模型自行拒绝”和“Agent-Firewall 实际阻断”的证据来源。

6 总结与展望

6.1 主要工作总结

本文基于 Agent-Firewall 系统，完成了面向 OpenClaw 工具调用智能体的中间安全层 Web 应用系统设计与实现。系统从 Proxy 文本安全和 Agent 能力安全两个维度建立边界：Proxy Firewall 通过本地顺序流水线对模型调用前输入进行解析、意图识别、规则/扫描器检查、风险聚合和审计；Agent Runtime 通过 graph-compatible 运行器对工具调用前后的权限、参数、上下文、输出和委派链进行控制。系统前端提供攻击演练、人工审批、OpenClaw 绑定、Trace 可视化和运行时诊断，使安全决策不仅可执行，而且可解释、可复核。

本文的主要成果包括：

(1) 建立了适用于 OpenClaw/MCP 类工具调用智能体的威胁模型，覆盖提示词注入、工具滥用、混淆代理、间接注入和委派链滥用。

(2) 实现了确定性风险聚合机制，将意图、规则、本地扫描器、PII、密钥和自定义规则统一映射到 ALLOW、MODIFY 和 BLOCK 决策。

(3) 实现了 Agent 工具调用前后双门控，包括 RBAC、参数 Schema、上下文风险、限额、确认流、PII/密钥清洗和间接注入阻断。

(4) 将子智能体创建和委派建模为受保护工具调用，使委派链可以被门控和 Trace 系统审计。

(5) 完成离线确定性复核与本机联调：离线测试覆盖核心权限、清洗逻辑、runtime spec 兼容和 11 类工具链回放；本机联调验证 OpenClaw 工具放行、工具确认、后置清洗、输入阻断和审批后重放。

6.2 创新点

本文的创新点主要体现在工程架构而非单一检测算法。第一，系统将 Agent 安全拆分为 Proxy 文本安全和 Agent 能力安全两个相互独立但可组合的层次，避免单一防火墙同时承担所有职责。第二，系统把工具调用前、工具调用后和模型调用前三个关键边界显式化，形成可审计的状态机，而不是依赖提示词约束。第三，系统把子 Agent 委派和 OpenClaw provider 调用都建模为受保护工具能力，使多 Agent 协作与本机 skill 执行进入同一套 RBAC、参数验证和 Trace 体系。第四，系统提供了离线可复现测试、

Agent 工具链回放与本机工具流联调三类证据，分别支持复现性验证、连续状态转移验证和本机链路验证。

6.3 局限性

系统仍存在若干局限。第一，本次轻量离线模式对多语言、编码、凭据变体和语义伪装攻击覆盖不足，依赖完整本地扫描器才能达到较高检出率。第二，本次真实联调主要验证了低风险摘要类 OpenClaw skill，尚不能推断文件、账号、支付或外部写操作 skill 具有同等安全性；这些工具在生产环境中仍应保持高敏或禁用。第三，post-tool gate 能够清洗工具输出中的 PII 和密钥，但如果用户原始输入已经包含敏感片段，最终回复仍可能复述这些输入内容，因此还需要更严格的输出侧策略和用户输入回显控制。第四，本次红队实验主要是单轮场景，尚未完整覆盖低慢多轮诱导和长期记忆污染。第五，系统仍以本地演示和毕业设计验证为主，管理 API 认证、长期 Trace 持久化、密钥管理和多租户隔离仍需按生产要求加强。

6.4 未来工作

后续研究可以从四个方向推进。第一，加入能力证明和工具描述签名，对 MCP/OpenClaw 工具的名称、描述、参数 Schema 和权限声明进行完整性验证，降低 Tool Poisoning 和 Rug Pull 风险。第二，建立多轮风险累计模型，将同一会话内的失败工具调用、意图漂移、角色声明变化和委派链扩张纳入风险评分。第三，强化子 Agent 策略验证，使主 Agent、子 Agent 和外部工具之间的权限边界可以静态检查和运行时证明。第四，完善生产化能力，包括认证授权、TLS、密钥管理、长期 Trace 数据库、告警策略和面向企业内部 Agent 平台的集成 SDK。

参考文献

- [1] Model Context Protocol. Model Context Protocol Specification[EB/OL]. [2026-05-08]. <https://modelcontextprotocol.io/specification>.
- [2] Model Context Protocol. Authorization - Model Context Protocol[EB/OL]. [2026-05-08]. <https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization>.
- [3] Deng X., Zhang Y., Wu J., et al. Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats[EB/OL]. arXiv:2603.11619, 2026[2026-05-08]. <https://arxiv.org/abs/2603.11619>.
- [4] Wang Z., Tu H., Zhang L., et al. Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw[EB/OL]. arXiv:2604.04759, 2026[2026-05-08]. <https://arxiv.org/abs/2604.04759>.
- [5] Wang Y., Gao H., Niu Z., et al. A Systematic Security Evaluation of OpenClaw and Its Variants[EB/OL]. arXiv:2604.03131, 2026[2026-05-08]. <https://arxiv.org/abs/2604.03131>.
- [6] Maloyan N., Namiot D. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents[EB/OL]. arXiv:2601.17549, 2026[2026-05-08]. <https://arxiv.org/abs/2601.17549>.
- [7] Huang C., Huang X., Tran N. P., Fard A. M. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning[EB/OL]. arXiv:2603.22489, 2026[2026-05-08]. <https://arxiv.org/abs/2603.22489>.
- [8] Jamshidi S., Nafi K. W., Dakhel A. M., et al. Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks[EB/OL]. arXiv:2512.06556, 2025[2026-05-08]. <https://arxiv.org/abs/2512.06556>.
- [9] OWASP Foundation. OWASP Top 10 for Large Language Model Applications[EB/OL]. [2026-05-08]. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [10] FastAPI. FastAPI Documentation[EB/OL]. [2026-05-08]. <https://fastapi.tiangolo.com/>.
- [11] LiteLLM. LiteLLM Documentation[EB/OL]. [2026-05-08]. <https://docs.litellm.ai/>.
- [12] Protect AI. LLM Guard Documentation[EB/OL]. [2026-05-08]. <https://llm-guard.com/>.
- [13] Microsoft. Presidio: Data Protection and De-identification SDK[EB/OL]. [2026-05-08]. <https://microsoft.github.io/presidio/>.
- [14] NVIDIA. NeMo Guardrails Documentation[EB/OL]. [2026-05-08]. <https://docs.nvidia.com/nemo/guardrails/>.

- [15] OpenClaw. Gateway CLI Documentation[EB/OL]. [2026-05-08].
<https://docs.openclaw.ai/cli/gateway>.
- [16] OpenClaw. Telegram Channel Documentation[EB/OL]. [2026-05-08].
<https://docs.openclaw.ai/channels/telegram>.
- [17] Agent-Firewall Project. Architecture Documentation[Z]. 2026:
docs/architecture/ARCHITECTURE.md.
- [18] Agent-Firewall Project. Agent Runtime Pipeline Documentation[Z]. 2026:
docs/architecture/AGENT_PIPELINE.md.

致谢

本论文与系统实现得以完成，首先感谢指导教师杨波教授在选题、研究方向和论文规范方面给予的指导。感谢北京林业大学工学院提供的本科毕业论文训练环境，使我能够把 Web 应用开发、网络安全、智能体系统和工程实验结合起来完成一次较完整的系统设计与研究。

感谢开源社区提供的 FastAPI、Nuxt、Vuetify、LiteLLM、LLM Guard、Presidio、NeMo Guardrails 等工具。它们使本项目能够在有限时间内聚焦于 Agent 安全架构、工具调用门控和实验验证，而不是重复实现基础框架。

感谢家人、同学和朋友在毕业设计期间给予的理解与支持。论文中仍有不足之处，后续将继续围绕工具调用智能体安全、协议级能力证明和多 Agent 委派治理展开改进。

附录 A 复现实验命令

A.1 Agent 层纯内存测试

```
cd /Users/isaachuo/Agent-Firewall/apps/agent  
  
PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest \  
tests/test_pre_tool_gate.py tests/test_post_tool_gate.py \  
tests/test_rbac.py tests/test_graph.py tests/test_chat_endpoint.py -q
```

本次复测输出为:

```
132 passed in 0.14s
```

OpenClaw 工具导入、角色配置和 runtime spec 兼容性使用如下命令复核:

```
cd /Users/isaachuo/Agent-Firewall/apps/proxy-service  
  
PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest \  
tests/control_plane/test_openclaw_tools.py \  
tests/control_plane/test_tools_roles.py -q
```

本次复测输出为:

```
59 passed in 17.70s
```

A.2 Agent 工具链离线回放

连续工具调用、前置拦截、审批停顿、后置清洗和 Trace 证据链使用离线脚本复核:

```
cd /Users/isaachuo/Agent-Firewall  
  
PYTHONPATH=apps/agent apps/agent/.venv/bin/python \  
scripts/evaluation/run_agent_chain_cases.py \  
--output-dir article/latex/generated --strict
```

该脚本不调用真实 Telegram、OpenClaw、MCP、外部 LLM 或 HTTP 服务,只复用 Agent Runtime 中的门控与 Trace 实现,并在执行阶段注入脱敏合成工具结果。输

出 文 件 包 括 `article/latex/generated/agent_chain_cases.json` 和 `article/latex/generated/agent_chain_cases_table.tex`; 后者由正文第 5 章直接\input 引用。严格模式下只要任一案例的 `pre-tool` 决策、`post-tool` 决策或实际执行工具集合不符合预期, 脚本就以非零状态退出。本次复测输出为:

```
Agent chain replay: 11/11 passed
```

A.3 红队场景离线复测

```
cd /Users/isaachuo/Agent-Firewall/apps/proxy-service

PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest
tests/test_scenario_coverage.py \

tests/test_scenario_deterministic.py -q
```

该命令读取 `apps/proxy-service/data/scenarios/` 中的 358 个场景, 并在 `pre-LLM` 流水线口径下检查预期决策。本次输出为 78 failed, 470 passed, 125 xfailed in 3.76s, 其中失败项作为正文中轻量规则与扫描器覆盖边界的复核证据。

A.4 执行耗时复核

```
cd /Users/isaachuo/Agent-Firewall/apps/proxy-service

PYTHONDONTWRITEBYTECODE=1.venv/bin/python -m pytest tests/test_graph.py
tests/test_scenario_deterministic.py \

--durations=10 -q
```

该命令用于列出耗时最高的测试项; 正文不再引用系统源文件中不存在的独立 `benchmark` 模块。

A.5 真机联调脱敏复核

本机联调启动 `Proxy`、`Agent Runtime` 和 `Telegram Bridge` 后进行, 服务端不记录或输出口凭据、真实会话标识和真实用户标识。复核时先检查运行时状态:

```
curl http://127.0.0.1:8000/health

curl http://127.0.0.1:8002/health

curl http://127.0.0.1:8002/agent/openclaw/config
```

```
curl http://127.0.0.1:8000/v1/agents/<agent-id>/runtime-spec
```

确认流复核时仅临时把 `openclaw_summarize` 调整为 `high` 且 `requires_confirmation=true`，触发 `tool_confirmation` 后批准并携带 `approved_intervention_id` 重放；复核结束后恢复为 `medium` 且不强制确认。脱敏证据只统计 `Trace`、`request`、`intervention` 数量和决策状态，不写入 Telegram 凭据、真实会话标识或原始敏感消息。

附录 B 图表生成

本文 LaTeX 正文中的结构图、流程图和示意图由 `article/latex/generate_autofigure_figures.py` 管理。该脚本会生成每张图的 AutoFigure-Edit 图稿说明，并默认使用 Researcher 风格的本地确定性渲染器输出可编辑 SVG、预览 PNG 和 LaTeX 引用 PDF；在配置 `AUTOFIGURE_EDIT_DIR`、`OPENAI_API_KEY` 和兼容/v1 的 `AUTOFIGURE_BASE_URL` 后，也可调用外部 AutoFigure-Edit CLI 进行模型辅助重绘。数值图不使用 AutoFigure 生成，仍由 `article/latex/generate_figures.py` 基于本地场景资产和论文固定口径绘制。推荐复现命令如下：

```
cd /Users/isaachuo/Agent-Firewall

python3 article/latex/generate_autofigure_figures.py --preview-contact-sheet

python3 article/latex/generate_figures.py
```

若需要调用真实 AutoFigure-Edit，可先在仓库外部克隆该工具，并设置环境变量：

```
export AUTOFIGURE_EDIT_DIR=/path/to/AutoFigure-Edit

export OPENAI_API_KEY=<redacted>

export AUTOFIGURE_BASE_URL=https://your-openai-compatible-provider.example/v1

export AUTOFIGURE_REFERENCE_IMAGE_PATH=/path/to/style-reference.png

python3 article/latex/generate_autofigure_figures.py --use-reference-image --force-autofigure
```

其中 `AUTOFIGURE_REFERENCE_IMAGE_PATH` 用于指定目标风格参考图，`--preview-contact-sheet` 会额外生成九张结构图的总览 PNG，便于人工目检。本文结构图的提示词按 Researcher/CycleResearcher 式论文流程图风格组织：米白背景、虚线外框、彩色分区、左侧角色图标块、文件堆与工具图标、茶金色粗箭头和紧凑节点卡片；未配置真实 AutoFigure-Edit 时，本地确定性渲染器也按同一风格生成可编译 PDF。

论文 DOCX 定稿链路中的核心结构图与统计图由 `article/docx/generate_publication_figures.py` 生成。该命令用于 DOCX 定稿链路，与本文 LaTeX 图表不是同一生成路径。该脚本使用 Matplotlib、SciencePlots 和 sciplot 兼容构建环境，红队分布直接读取 `apps/proxy-service/data/scenarios/` 中的场景 JSON，输出 300 dpi 论文风格 PNG 至 `article/docx/generated_figures/`；DOCX 构建脚本会在生成正文前自动刷新这些图像。推荐复现命令如下：

```
cd /Users/isaachuo/Agent-Firewall/article/docx

.thesis-venv/bin/python generate_publication_figures.py

/Users/isaachuo/.cache/codex-runtimes/codex-primary-
runtime/dependencies/python/bin/python3 \

build_thesis_docx.py
```

附录 C 关键代码文件索引

路径	作用
apps/proxy-service/src/pipeline/graph.py	Proxy Firewall 本地顺序流水线，保留 graph 兼容接口。
apps/proxy-service/src/pipeline/nodes/decision.py	风险评分与 ALLOW/MODIFY/BLOCK 决策。
apps/agent/src/agent/graph.py	Agent Runtime graph-compatible 顺序运行器。
apps/agent/src/agent/nodes/pre_tool_gate.py	工具调用前置门控。
apps/agent/src/agent/nodes/post_tool_gate.py	工具结果后置清洗与阻断。
apps/agent/src/agent/rbac/rbac_config.yaml	默认角色、工具权限、敏感度与确认配置。
apps/agent/src/agent/tools/hub.py	Internal、MCP、OpenClaw 和子 Agent 委派统一执行入口。
apps/agent/src/agent/subagents.py	子 Agent 创建与 OpenClaw 委派运行时。
scripts/evaluation/run_agent_chain_cases.py	Agent 工具链离线回放与论文表格生成脚本。
article/latex/generate_autofigure_figures.py	LaTeX 结构图、流程图和示意图的 AutoFigure-Edit 提示词与 PDF 生成脚本。
article/latex/generate_figures.py	LaTeX 数值图和结构化分析图生成脚本。
apps/proxy-service/src/control_plane/services/runtime_spec.py	Control Plane runtime spec 构建与旧 skill metadata 归一化。
apps/proxy-service/data/scenarios/	红队场景数据。
docs/architecture/	系统架构、运行时流水线和威胁模型文档。